

Problem 1 — Answering the requirements directly (indicative numbers)

Requirement	Naive fully-automated SDL	This proposal (two-tier, human-in-the-loop)
High-throughput	Bottlenecked anyway: probing/anneal robotics limit end-to-end rate	Fast tier: 10–15 structural fingerprints/day + in-situ data every cycle (runs 24/7 unattended). Expensive tier: ~2x fewer electrical tests per insight (shown in silico) — effective throughput is measured in insights/month, not wafers/day
Footprint	150–250 m2 vacuum-integrated cluster + custom transfer robotics	One lab bay, ~60–80 m2: ALD + in-situ ellipsometry, XRR/GIXRD, RTP, probe station — all standard commercial tools, no custom integration
Cost	\$8–15M class, 3+ years of robotics integration before first value (cf. Ada: ~\$8M for one property)	\$1.5–2.5M capex + 2–3 FTE. No bespoke robotics in Phases 0–1; automation added in Phase 2 only where data proves it pays
Period	Value arrives only after full integration succeeds	Phase 0 (6 mo): decision engine validated on existing/retrospective data. Phase 1 (by month 12–18): first virtual-metrology model + mapped process window — each phase delivers standalone value

Numbers are indicative planning estimates for discussion (tool list & vendor quotes in backup) — the design choice they reflect: spend capital on measurements and intelligence, not on robotizing steps a human does better during R&D.

Problem 1 — From SDL to the fab: what Samsung gets beyond one material

The SDL's product is not a hero film — it is a **TRANSLATOR** (fast optical/structural data -> electrical performance) plus a map of where that translator is valid. That is fab language: virtual metrology.

1 · R&D acceleration (now)

- **2x fewer electrical-test cycles** to a mapped process window (in-silico result) — directly shortens dielectric screening campaigns
- **Candidates arrive with confidence labels**, not just best guesses — fewer surprises downstream

2 · Pilot-line transfer (next)

- **The trained translator deploys** against inline ellipsometry/XRR that production lines already run on every wafer
- **Validity map tells engineers** WHEN inline data can replace destructive electrical/TEM sampling — and when it cannot (risk control, not just prediction)

3 · Compounding data asset

- **Every sample carries full provenance:** recipe -> tool logs -> fingerprint -> anneal -> electrical
- **Dataset + models transfer** to the next dielectric program (doped HfO₂, FE-HZO, new precursors) — the asset outlives any single target spec

Where it lands (examples)

- **DRAM capacitor dielectrics:** EOT scaling needs new doped ZrO₂/HfO₂ stacks every node
- **GAA / advanced logic gate stacks:** interface + leakage co-optimization
- **FE-HZO memory:** wake-up/endurance vs anneal — same two-tier problem
- **Same engine, new objectives:** only the metrology list changes

KPI view (how I would be measured): electrical-test wafers per learned process window (target: -50%); time from 'new candidate material' to 'mapped safe operating window' (target: months, not years); fraction of routine electrical tests replaced by trusted virtual metrology (target grows every quarter); dataset reuse across programs (second program should start at ~30% lower cost).

Problem 1 — Risks I have designed for (and what stays honest)

Tool drift & metrology noise can mimic physics

Scheduled reference depositions + 2 process replicates per batch; noise is an explicit, learned term in the model. In the benchmark, removing replicates collapsed hidden-zone detection to chance — this is load-bearing, not bookkeeping.

Probe/anneal automation is where thin-film SDLs die

Human-in-the-loop tier + pre-patterned test substrates / Hg probe from day one. Robotics only in Phase 2, only where Phase-1 data proves the ROI.

Lab test structure is not the device stack

Position outputs as candidate windows; Phase-2 transfer validation on device-representative stacks before any claim reaches a product team.

Algorithm can lock in on a wrong 'untrustworthy' label

Found in silico: an early wrong noise estimate can freeze exploration. Fixed with an exploration safeguard (one random pick per batch). Cost to find it in simulation: days. In the lab: months.

Deposition temperature changes are slow (30-60 min stabilization)

Cost-aware scheduling batches experiments by thermal budget — the optimizer pays a modeled price for temperature moves.

Problem 2 — The three requirements map to three architectural decisions

REQ 1 · Individual AND integrated operation

- **Hierarchical control planes:** device layer -> per-SDL orchestrator -> fleet layer. Each SDL is fully functional standalone; the fleet layer only coordinates.
- **Shared modules (robot arms, furnaces, analyzers)** are RESOURCES with a reservation-based scheduler — no workflow owns a device, it books capabilities.
- **Graceful degradation:** if the fleet layer is down, every SDL keeps running its local queue. Integration is additive, never a dependency.
- **Fab analogy:** this is MES + equipment-level control, the pattern Samsung already trusts at scale — applied to research labs.

REQ 2 · Usable by non-SDL experts

- **Researchers compose CAPABILITIES, not devices:** a no-code canvas of verbs ('anneal at 500 C', 'measure thickness') — the scheduler resolves which physical tool executes.
- **Recipe templates per domain** (thin film, battery, organic): start from a working experiment, edit parameters, not code.
- **Dry-run by default:** every workflow executes first against the digital twin (simulated devices) — errors surface before hardware moves.
- **Safety interlocks & approval gates** are declared in the device contract, enforced by the orchestrator — not by user discipline.

REQ 3 · HW add/change/remove, minimal burden

- **Device-plugin contract:** a containerized driver + a machine-readable capability descriptor (verbs, parameter ranges, units, safety limits, calibration state).
- **Register -> everything updates automatically:** scheduler sees a new resource, GUI auto-generates its panel, provenance logging attaches — zero orchestrator code changes.
- **Workflows bind to capabilities, not device IDs:** swapping a furnace vendor is a config change; no workflow is rewritten.
- **Versioned configs + twin regression test** before a changed device re-enters production use.

One sentence: standardize the CONTRACT (capability descriptor + communication grammar), containerize the drivers, centralize only scheduling and data — then labs scale like fleets, not like one-off projects.

Problem 2 — 'Minimal burden' made concrete: a new instrument joins in ~1 day

1 · Wrap

- Vendor driver goes into a standard SDK container — whatever the tool speaks (RS-232, Modbus, TCP, vendor DLL), the container's outward face is the universal grammar.

2 · Declare

- Capability descriptor (YAML): verbs it offers, parameter ranges & units, timing, safety limits, consumables, calibration schedule. This file IS the integration.

3 · Register

- Container announces itself to the orchestrator. Scheduler now sees a new bookable resource; data layer attaches provenance logging automatically.

4 · Verify

- Digital-twin dry-run + supervised first runs (shadow mode). Device is 'trusted' only after passing — same discipline as fab equipment qualification.

5 · Use

- GUI panel auto-generated from the descriptor; existing workflows that request matching capabilities can now be scheduled onto it. No workflow rewrites anywhere.

Why this is the industrially credible answer: integration cost is the #1 reason lab-automation programs stall. Making the capability descriptor the single integration artifact converts 'integration projects' (weeks of bespoke code, one engineer per instrument) into 'configuration work' (one file + one container). It also future-proofs the fleet:

removal is just deregistration (scheduler routes around it); **replacement** is a descriptor diff; **upgrades** are versioned and twin-tested before re-entering service — the same lifecycle discipline fabs apply to production equipment, scaled down to research.

Problem 2 — The two hard problems under the hood: shared-resource scheduling & data

Scheduling shared modules (the real multi-SDL problem)

- **Reservation-based allocation:** workflows request capability + time window; the scheduler books physical resources — shared robot arms and furnaces stop being collision points.
- **Priorities & preemption policy:** campaign deadlines, maintenance windows, and calibration runs coexist by policy, not by hallway negotiation.
- **Deadlock avoidance:** all-or-nothing resource acquisition per workflow step (no workflow holds the robot while waiting for a busy furnace).
- **Fairness across SDLs:** per-lab budgets/quotas so one hot project cannot starve the others.
- **Failure containment:** a faulted device is quarantined; queued steps re-route to equivalent capabilities or pause cleanly — the fleet never cascades.

One data backbone for every SDL (thin film, battery, organic...)

- **Sample-centric provenance graph:** recipe -> device logs -> measurements -> model decisions — one queryable record per sample, across labs and modalities.
- **Shared schema, domain-specific vocabularies:** units, materials, and method ontologies per domain plug into one common core — integration without forcing uniformity.
- **Streaming telemetry with drift alarms:** FDC-style monitoring for research tools (reference-run tracking catches drift before it poisons a campaign).
- **Decisions are data too:** every algorithm suggestion is logged with model version + inputs — full auditability, reproducible campaigns, and the training corpus for the next generation of models.
- **This is the asset that compounds:** hardware depreciates; the provenance-complete dataset and the models trained on it appreciate.

Positioning: the GUI and microservices make the system usable; the scheduler and the data backbone make it an INSTITUTIONAL capability — multiple labs, one operational discipline, one growing asset. That is the difference between a lab demo and infrastructure a company runs for ten years.